

[Portfolio] Eventer Map: 知能型公演・イベント情報統合サービス

1. プロジェクト概要 (Overview)

「すべての公演とイベント、これからは地図で一目で確認しましょう。」

Eventer Mapは、散らばっている公演およびイベント情報を地図を中心に視覚化し、ユーザーが希望する時間と場所のイベントを直感的に探索・管理できるよう支援するウェブアプリケーションです。

- 開発期間: 2025.11 ~ (持続的に高度化中)
- 主な役割: プロジェクト企画、システム設計、フルスタック開発およびインフラ構築
- 核心价值: データ正規化による正確な情報の提供、位置ベースの直感的なUX、低コスト・高効率な安定したセルフホスティングインフラ

2. 主な機能と例 (Key Features & Examples)

□ Google Mapsベースのインタラクティブな探索

- リアルタイムの位置ベースのイベントマーカー表示および詳細情報(InfoWindow)の提供

- 「いつ、どこで」開催されるイベントかを地図上で直感的に把握可能

📅 日付選択

2026/05/30 📅

🔍 出演者検索

🔍 出演者名または別名を入力

+ 新規イベント登録

イベント一覧 2

ゆず

📍 長崎スタジアムシティ HAPPINESS ARENA

🕒 時間未定

🔍 ゆず

SHIGURE UI Birthday Live "Wishing Umbrella" 三次先行

📍 Kアリーナ横浜

🕒 時間未定

野村

雲取山

丹

大月線 都留

富士吉田市

忍野村

山中湖村



 Create Event View

- **[例]:**「2026-03-31」の日付フィルターを選択すると、その日に開催される東京ドームや渋谷近辺のすべての公演情報が地図上に即座に同期して表示されます。

□ 知能型データの正規化および管理

- **出演者/場所の自動正規化:** 様々な表記法を一つの標準名称に統合管理し、データの整合性を確保
- **ユーザー入力例:**
 - IVE、ive、アイヴ、ヨアソビ -> システム内部で**ive、yoasobi**のような単一の固有値として自動マッチングおよび保存
- **別名(Aliases)システム:** 「東京ドーム」検索時、「Tokyo Dome」、「Tokyo Dome」など多言語の別名に基づいた検索をサポート



- **ハイブリッドインフラの活用:** 高性能な推論が必要な**Event Extractor**と**SGLang**はデスクトップ(GPU/WSL2)環境で、ウェブサービスとDBは**Synology NAS**で駆動させる効率的な分散アーキテクチャを構築

□ ユーザー認証および参加システム

- **Google OAuth 2.0:** ソーシャルログインによる簡便で安全な認証体系
- **パーソナライゼーション(Personalization)の例:** ユーザーが特定のアーティストの公演で「参加予定(Join)」ボタンをクリックすると、そのイベントはユーザーの個人ダッシュボードに即座に反映され、地図のフィルタリング時に優先的に露出されます。

□ 安定したデータ履歴管理

- すべてのイベント作成/修正/削除活動を履歴として記録し、データの変更を追跡可能(Audit logging)
- **[例]:** 特定のイベントの通報が5回以上累積された場合、管理者が介入しなくてもシステムが自動的にそのイベントをis_hidden=True処理し、コミュニティの浄化機能を遂行します。

3. 技術スタック (Tech Stack)

Frontend

- **Core:** React, TypeScript, TanStack Query (React Query)
- **Styling:** Tailwind CSS, shadcn/ui

Backend

- **Framework:** FastAPI (Python 3.11/3.12), SQLAlchemy 2.0, Alembic
- **AI & Pipeline:** SGLang, Qwen2.5-7B-Instruct (GPTQ-Int4), Gmail API

Infrastructure & DevOps

- **Hardware (Desktop):** NVIDIA RTX 3060 (12GB VRAM)
 - **Hardware (Server):** Synology NAS (Self-hosting, INTEL Celeron J4125, 20GB RAM)
 - **Environment:** Docker, NVIDIA Container Toolkit (WSL2), Nginx, PostgreSQL
-

4. 核心的な技術的挑戦と解決 (Technical Achievements)

□ 高精度な重複防止システムの構築

- 問題: ユーザーが同一のイベントを重複して登録したり、類似した名称でデータが断片化したりする問題が発生
- 解決: **Jaccard係数(Jaccard Similarity)と測地線距離(Geodesic Distance)**を結合したハイブリッド重複検出ロジックを実装
- 核心アルゴリズムコード (Jaccard Similarity):

```
def calculate_performer_similarity(event1, event2):  
    """出演者リスト間の積集合/和集合の比率による類似度測定"""  
    p1 = set(p.normalized_name for p in event1.performers)  
    p2 = set(p.normalized_name for p in event2.performers)  
  
    if not p1 or not p2: return 0.0  
  
    intersection = len(p1 & p2)  
    union = len(p1 | p2)  
  
    return intersection / union if union > 0 else 0.0
```

⚠️ 重複の可能性を検出

1個の類似イベントが見つかりました。

SHIGURE UI Birthday Live "Wishing Umbrella" 三次先行

📍 **会場**
Kアリーナ横浜

📅 **日付**
2026-05-30

詳細一致情報

タイトル

出演者

距離

✏️ このイベントを編集する

キャンセル

このまま登録する

- **成果:** 日付(25%)、距離(20%)、出演者(25%)、タイトル(15%)などに重みを付与した総合スコアの算出により、データの整合性を90%以上向上させました。

□ ハイブリッドAIインフラの構築 (NAS + Desktop GPU)

- **問題:** 高性能LLM(Qwen2.5-7B)推論のためにGPU加速が必須だが、低電力ベースのSynology NAS単独では性能に限界が存在
- **解決:** デスクトップ(Windows 11 + WSL2)のRTX 3060リソースを活用して**SGLang**サーバーと**Event Extractor**を個別に駆動し、結果データのみNASのDBに連動させるハイブリッド構造を設計
- **成果:** 既存インフラの限界を克服し、高性能AI機能をサービスに統合。外部APIコストなしでリアルタイムデータ抽出パイプラインを完成
- **核心プロンプト設計 (Prompt Engineering):**

非定型データの精製およびJSON抽出のためのシステムプロンプトの例

_SYSTEM_PROMPT = ""

1. Extract:

- title: STRICT: Exclude ticket sales types (e.g., "先行", "一般発売").
- performers: list of artists. STRICT: Exclude roles (e.g., "東山奈央(志摩リン役)").
- location: Name of the venue ONLY. Do NOT include region (e.g., "(東京都)").

2. Always respond with ONLY valid JSON format.

"""

□ 低コスト・高効率な安定的インフラ運営

- 問題: 動的IP環境で外部接続の安定性を確保し、Google Maps APIの呼び出しコストを管理しなければならない課題
- 解決: MyDNSとSynology Reverse Proxyを連動させ、ジオコーディングの結果をDBにキャッシュする構造を設計

システムアーキテクチャ (Infrastructure)

```
graph TD
    User["□ ユーザー (Browser)"] -- HTTPS/443 --> MyDNS["□ MyDNS (DDNS)"]
    MyDNS -- Forward --> NAS_Host["□ Synology NAS"]

    subgraph NAS_Node ["□ Synology NAS (Web Node)"]
        Proxy["□ リバースプロキシ"]
        Nginx["□ Nginx (Frontend)"]
        App["* FastAPI Backend"]
        DB["□ PostgreSQL"]
    end

    subgraph Desktop_Node ["□ Desktop (AI Node)"]
        Extractor["□ Event Extractor (WSL2)"]
        SGLang["□ SGLang (Docker)"]
        GPU["□ NVIDIA RTX 3060 (WSL)"]
    end

    subgraph External ["外部連携"]
        Gmail["□ Gmail API"]
    end

    NAS_Host --> Proxy
    Proxy --> Nginx
    Nginx --> App
    App --> DB

    Extractor -- "Fetch Email" --> Gmail
    Extractor -- "Inference" --> SGLang
    SGLang -- "Qwen" --> GPU
    Extractor -- "Sync Data" --> App
```

5. Technical Deep Dive: GPU加速ベースのAIインフラ

NVIDIA Container Toolkit (GPU Passthrough)

- 役割: DockerコンテナがホストのハードウェアGPUに直接アクセスできるようにする架け橋の役割を果たします。
- 意義: 複雑なドライバー設定なしに、コンテナ環境でSGLangのようなAI推論エンジンがGPUの並列演算能力を100%活用できるようにします。

RTX 3060 (WSL2) ハイブリッド環境

- **技術的調和:** Windowsの便利な操作性とLinuxの強力な開発エコシステム(WSL2)を結合した分散環境です。
- **効率性:** Linuxカーネルから直接GPUリソースをパススルー(Passthrough)し、AIライブラリが最適化されたLinux環境で最高のパフォーマンスで推論を遂行できるように設計しました。

Qwen (Qwen2.5-7B) モデルの選定理由

- **言語理解度:** 日本語や韓国語などのアジア圏の言語に対する理解度が非常に高く、多言語の公演メールの精密な分析に最適化されています。
- **性能対効率:** RTX 3060(12GB VRAM)一枚でも円滑に駆動可能な7Bサイズでありながら、商用モデルに匹敵する優れた指示遵守(Instruction Following)能力を備えています。

6. プロジェクトの成果とインサイト

1. **データの整合性の体系的な確保:** 単なる開発を超え、実際の運営時に発生するデータの断片化問題をアルゴリズムで解決し、バックエンド設計能力を証明
2. **フルスタックの観点からの問題解決:** フロントエンドの直感性からインフラの可用性設定まで、全体のパイプラインを直接構築し、総合的なエンジニアリング能力を保有